

Architecture Patterns for AI Applications

Module 16 — Business AI in Practice · Notebook 50

Prof. Dr. Christoph Weisser

HSBI

Summer Semester 2026

2026-07-22

Architecture Patterns for AI Applications

Architecture Patterns for AI Applications
Module 16 — Business AI in Practice · Notebook 50

Prof. Dr. Christoph Weisser

HSBI

Summer Semester 2026

Contents

- 1 Traffic
- 2 Patterns
- 3 Frontend & backend
- 4 Pipeline
- 5 Choosing
- 6 Course map

2026-07-22

Architecture Patterns for AI Applications

└ Contents

Contents	
1	Traffic
2	Patterns
3	Frontend & backend
4	Pipeline
5	Choosing
6	Course map

Outline

- 1 Traffic
- 2 Patterns
- 3 Frontend & backend
- 4 Pipeline
- 5 Choosing
- 6 Course map

2026-07-22

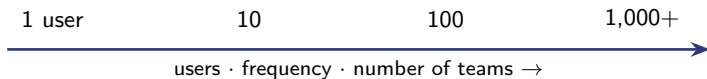
Architecture Patterns for AI Applications

- └ Traffic
 - └ Outline

Outline

- 1 Traffic
- 2 Patterns
- 3 Frontend & backend
- 4 Pipeline
- 5 Choosing
- 6 Course map

Architecture is the answer to a *traffic* question



Ask three things before you choose

- **How many users**, and growing how fast?
- **Who invokes it** — a human, a schedule, or another service?
- **How many teams** will own and ship pieces of it?

Architecture is not sophistication — it is matching *structure* to *load*.

2026-07-22

Architecture Patterns for AI Applications

└ Traffic

└ Architecture is the answer to a *traffic* question

Architecture is the answer to a *traffic* question



Ask three things before you choose

- How many users, and growing how fast?
- Who invokes it — a human, a schedule, or another service?
- How many teams will own and ship pieces of it?

Architecture is not sophistication — it is matching structure to load

~8 min. Point: architecture is a response to *load*, not a measure of skill. Set that frame now or the whole deck reads as a ladder to climb.

The three questions are the actual deliverable of this slide — users, invoker, team count. Notice the third is organisational: architecture tends to mirror team boundaries, which is why a two-person team should almost never build microservices.

Ask the room: “Your churn model runs once a month for one analyst. What architecture?” A script. Let someone argue for an API and then ask who would call it.

The failure mode to name early: engineers choose architecture by what is interesting, not by load.

Outline

- 1 Traffic
- 2 Patterns
- 3 Frontend & backend
- 4 Pipeline
- 5 Choosing
- 6 Course map

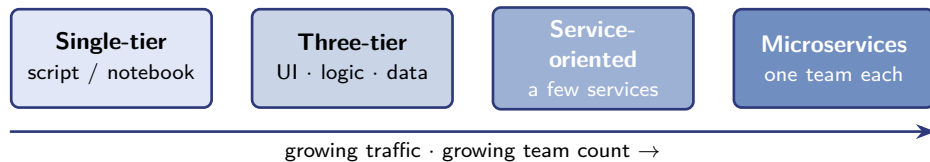
2026-07-22

Architecture Patterns for AI Applications

- └ Patterns
 - └ Outline

- Outline
- Traffic
 - Patterns
 - Frontend & backend
 - Pipeline
 - Choosing
 - Course map

Four patterns of growing complexity



Read left to right as *growing load*, not "primitive to advanced". The simplest workable choice wins.

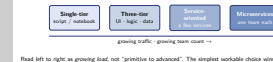
2026-07-22

Architecture Patterns for AI Applications

Patterns

Four patterns of growing complexity

Four patterns of growing complexity



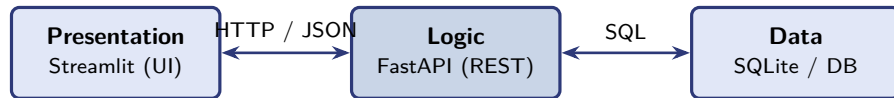
~8 min. Point: four shapes, increasing in operational cost. Read the footnote aloud — students instinctively read the arrow as a maturity ladder and conclude their notebook is embarrassing.

Give the one-line trigger for each: single-tier until someone else needs it; three-tier when a non-developer must use it; services when a *second team* needs to ship independently; microservices when many teams do.

Every step right costs you deployment, monitoring, network failure modes and debugging across process boundaries. Nothing is free, and none of it is model work.

Most of this course — and most real analytics work — lives happily in the first two boxes.

Three-tier — the workhorse for interactive AI features



- Each tier can change independently; the **REST contract** is the boundary.
- The API is reusable — a second UI, a CLI, or another team can call it.
- *You build exactly this in **Notebook 34 (POC 2)**.*

2026-07-22

Architecture Patterns for AI Applications

Patterns

Three-tier — the workhorse for interactive AI features



- Each tier can change independently; the **REST contract** is the boundary.
- The API is reusable — a second UI, a CLI, or another team can call it.
- You build exactly this in **Notebook 34 (POC 2)**.

~10 min — the workhorse pattern; spend time here.

Point: three tiers, and the **REST contract** is the real boundary. Once the contract is fixed, the tiers can be rewritten independently — that is the entire benefit.

The reusability bullet is the one that convinces people: the same API serves the Streamlit UI, a CLI, a scheduled job, and another team, at no extra cost. A notebook serves exactly one consumer.

Ask the room: “Why can’t the UI just query the database directly?” Because then business logic lives in two places and drifts — and every new consumer re-implements it. That is the whole argument for a logic tier.

Promise them NB 34: they build this exact shape, Streamlit + FastAPI + SQLite.

The over-engineering trap

Microservices for a 5-user pilot

Microservices became fashionable around 2015; many teams adopted them *before* they had the team size or traffic to justify the operational cost. The right unit of a service is a *capability* — not every Python module.

The rule

Pick the architecture you'll have *outgrown in 12 months*, not the one you'll *need in 36*.
Premature complexity kills more projects than migrations do.

2026-07-22

Architecture Patterns for AI Applications

Patterns

The over-engineering trap

~8 min. Point: the 12-month rule is the takeaway of the section. Say it twice.

The reasoning: you cannot predict 36 months, migrations are far cheaper than people fear, and complexity you don't need yet is paid for every single day in the meantime.

The unit-of-service line matters: a service should be a *capability* ("scoring", "ingestion"), not a Python module. Teams that split by file end up with a distributed monolith — all the network cost, none of the independence.

Ask the room: "What's the cost of being wrong in each direction?" Under-engineering is a bad week when you migrate; over-engineering is a bad year, every year.

The over-engineering trap

Microservices for a 5-user pilot

Microservices became fashionable around 2015; many teams adopted them *before* they had the team size or traffic to justify the operational cost. The right unit of a service is a *capability* — not every Python module.

The rule

Pick the architecture you'll have *outgrown in 12 months*, not the one you'll *need in 36*.
Premature complexity kills more projects than migrations do.

Outline

- 1 Traffic
- 2 Patterns
- 3 Frontend & backend
- 4 Pipeline
- 5 Choosing
- 6 Course map

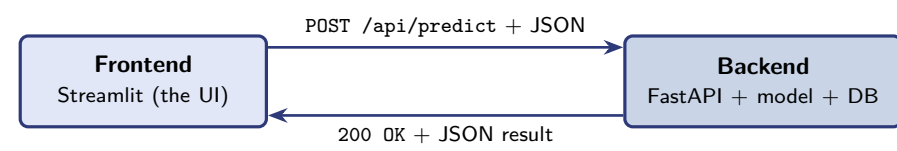
2026-07-22

Architecture Patterns for AI Applications

- └ Frontend & backend
 - └ Outline

- Outline
- Traffic
 - Patterns
 - Frontend & backend
 - Pipeline
 - Choosing
 - Course map

Frontend ↔ backend: the request/response cycle



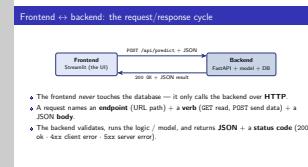
- The frontend *never* touches the database — it only calls the backend over **HTTP**.
- A request names an **endpoint** (URL path) + a **verb** (GET read, POST send data) + a JSON **body**.
- The backend validates, runs the logic / model, and returns **JSON** + a **status code** (200 ok · 4xx client error · 5xx server error).

2026-07-22

Architecture Patterns for AI Applications

└ Frontend & backend

└ Frontend ↔ backend: the request/response cycle



~**10 min.** Point: this is the mechanical picture behind “the tiers talk over HTTP.” For many students this is the first time client/server is made concrete.

Walk one request end to end, out loud: user clicks → frontend builds JSON → POST to the endpoint → backend validates → model runs → JSON back with 200 → UI renders. Then walk a *failure*: bad input → 422, model crash → 500.

The status-code distinction is worth drilling: 4xx means *you* sent something wrong, 5xx means *I* broke. Students conflate them constantly, and it determines who debugs.

First bullet is the security point: the frontend never touches the database. It runs on the user’s machine and cannot be trusted with credentials.

The backend — one FastAPI endpoint

```
from fastapi import FastAPI
from pydantic import BaseModel
```

```
app = FastAPI()
```

```
class Customer(BaseModel): # validates the incoming JSON
    tenure: int
    monthly_charges: float
```

```
@app.post("/api/predict") # endpoint path + HTTP verb
def predict(c: Customer):
    p = model.predict_proba([[c.tenure, c.monthly_charges]])[0][1]
    return {"churn_probability": round(float(p), 3)}
```

Pydantic **validates** the body automatically; returning a dict is serialised to JSON; interactive docs appear free at /docs.

2026-07-22

Architecture Patterns for AI Applications

└ Frontend & backend

└ The backend — one FastAPI endpoint

The backend — one FastAPI endpoint

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Customer(BaseModel): # validates the incoming JSON
    tenure: int
    monthly_charges: float

@app.post("/api/predict") # endpoint path + HTTP verb
def predict(c: Customer):
    p = model.predict_proba([[c.tenure, c.monthly_charges]])[0][1]
    return {"churn_probability": round(float(p), 3)}
Pydantic validates the body automatically, returning a dict is serialised to JSON; interactive docs appear free at /docs.
```

~8 min. Point: a real endpoint is about ten lines. Show that the ceremony they imagine isn't there.

Read it in the order the request arrives: the `Customer` class *is* the contract, the decorator binds path + verb, the body is ordinary Python, the returned dict becomes JSON.

The Pydantic point is the one worth dwelling on: you declared types, and got validation free — send a string where an int belongs and FastAPI returns a 422 with a precise message before your code ever runs. That is the 4xx from the previous slide, generated for you.

Demo /docs if you're live — interactive documentation generated from those type hints is the moment FastAPI sells itself.

The frontend — calling the backend

```
import requests, streamlit as st

resp = requests.post(
    "http://localhost:8000/api/predict",
    json={"tenure": 12, "monthly_charges": 79.9},
    timeout=10,
)
resp.raise_for_status() # turn 4xx / 5xx into an error
st.metric("Churn risk", resp.json()["churn_probability"])
```

- The frontend is just an **HTTP client** — the same requests pattern as NB 12.
- **CORS:** a browser blocks cross-origin calls — add FastAPI's CORSMiddleware so the UI (port 8501) may call the API (port 8000).

You build this end-to-end in **Notebook 34 (POC 2)** — Streamlit + FastAPI + SQLite.

2026-07-22

Architecture Patterns for AI Applications

└ Frontend & backend

└ The frontend — calling the backend

The frontend — calling the backend

```
import requests, streamlit as st

resp = requests.post(
    "http://localhost:8000/api/predict",
    json={"tenure": 12, "monthly_charges": 79.9},
    timeout=10,
)
resp.raise_for_status() # turn 4xx / 5xx into an error
st.metric("Churn risk", resp.json()["churn_probability"])

# The frontend is just an HTTP client — the same requests pattern as NB 12.
# CORS: a browser blocks cross-origin calls — add FastAPI's CORSMiddleware so the UI
# (port 8501) may call the API (port 8000).

You build this end-to-end in Notebook 34 (POC 2) — Streamlit + FastAPI + SQLite.
```

~8 min. Point: the frontend is *just an HTTP client* — the same requests they already met in NB 12. Nothing new is being learned; it's being reused.

`raise_for_status()` is the line students omit and then spend an hour debugging: without it a 500 sails through and you render an error page's body as data. Insist on it. Same for `timeout` — a hung request with no timeout hangs forever.

CORS will bite them in NB 34, so plant it now: the browser — not FastAPI — blocks the cross-origin call, which is why the error appears in the browser console and the server log looks clean. Knowing where to look saves the hour.

Ask the room: “The UI shows nothing and the server log looks fine. Where do you look first?”

Outline

- 1 Traffic
- 2 Patterns
- 3 Frontend & backend
- 4 Pipeline**
- 5 Choosing
- 6 Course map

2026-07-22

Architecture Patterns for AI Applications

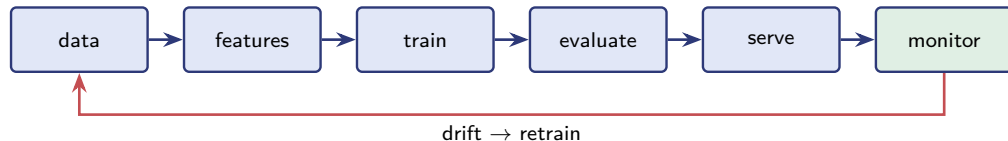
└ Pipeline

└ Outline

Outline

- Traffic
- Patterns
- Frontend & backend
- Pipeline**
- Choosing
- Course map

The cross-cutting ML / AI pipeline



What pure-software patterns don't capture

- **Training data is part of the artefact** — different snapshot, different product.
- **Drift exists** — 92 % in March is not 92 % in October.
- **The feedback loop has economic cost** — each prediction nudges the next training batch.

2026-07-22

Architecture Patterns for AI Applications

└ Pipeline

└ The cross-cutting ML / AI pipeline

The cross-cutting ML / AI pipeline



What pure-software patterns don't capture

- Training data is part of the artefact — different snapshot, different product.
- Drift exists — 92 % in March is not 92 % in October.
- The feedback loop has economic cost — each prediction nudges the next training batch.

~10 min — **why ML architecture isn't just software architecture.**

Point: the three bullets are what a software-architecture course would miss entirely.

“Training data is part of the artefact” is the one to labour: the same code with a different data snapshot is a different product, which is why reproducibility needs data versioning, not just Git.

The red drift arrow is the slide's spine — monitor feeds back to data. Software ships and is done; a model starts decaying on day one because the world moves. 92 % in March is not 92 % in October.

Connect forward: this loop is Module 13 (production) and NB 32 (observability); the question of *which* change moved the metric is Module 18.

The feedback bullet is subtle — if your predictions influence behaviour, they contaminate the next training set. Mention briefly; don't derail.

Outline

- 1 Traffic
- 2 Patterns
- 3 Frontend & backend
- 4 Pipeline
- 5 Choosing**
- 6 Course map

2026-07-22

Architecture Patterns for AI Applications

└ Choosing

└ Outline

- Traffic
- Patterns
- Frontend & backend
- Pipeline
- Choosing**
- Course map

Choosing the right size

Smells that you've outgrown your current shape

- “Can we run this nightly?” → add scheduling (single-tier job).
- A non-developer needs to use it → add a UI tier (three-tier).
- A second team wants to build on it → expose it as a service.

Two opposite mistakes

Over-engineering (microservices for a pilot) and **under-engineering** (a 200-user system held together by one notebook).

2026-07-22

Architecture Patterns for AI Applications

Choosing

Choosing the right size

Choosing the right size

Smells that you've outgrown your current shape

- “Can we run this nightly?” → add scheduling (single-tier job).
- A non-developer needs to use it → add a UI tier (three-tier).
- A second team wants to build on it → expose it as a service.

Two opposite mistakes

Over-engineering (microservices for a pilot) and **under-engineering** (a 200-user system held together by one notebook).

~8 min. Point: don't choose architecture up front — let *smells* tell you when you've outgrown the current shape.

Each smell maps to one move, and the move is small. That's the reassurance: you are never trapped, so choosing simple now is safe.

Give under-engineering equal air time. This deck spends more words on over-engineering, but a 200-user system running from one person's notebook is just as real a failure — and more common in analytics teams. The tell is a single point of failure wearing a lanyard.

Ask the room: “What's the smell that says you've outgrown a notebook?” Best answer: someone else needs to run it, or it must run when you're on holiday.

Outline

- 1 Traffic
- 2 Patterns
- 3 Frontend & backend
- 4 Pipeline
- 5 Choosing
- 6 Course map

2026-07-22

Architecture Patterns for AI Applications

- Course map
 - Outline

Outline

- Traffic
- Patterns
- Frontend & backend
- Pipeline
- Choosing
- Course map

How the course maps to these patterns

Pattern	Where it appears in the course
Single-tier (script / notebook)	Almost every notebook in the course, by default
Scheduled single-tier	NB 46 — scheduling & orchestration
Three-tier with AI in the middle	NB 34 (POC 2) and the NB 48 capstone
Service-oriented	llm_providers.py — pluggable providers
Microservices	<i>patterns only</i> — too big for a teaching course
ML pipeline	NB 17–19 + NB 45–46 — train / evaluate / serve
ML pipeline, deep-learning variant	NB 20–22 — train / regularise / export (TorchScript)

You *build* the three-tier and ML-pipeline shapes in **Notebook 34**; the rest you meet as patterns here.

2026-07-22

Architecture Patterns for AI Applications

- └ Course map
- └ How the course maps to these patterns

Pattern	Where it appears in the course
Single-tier (script / notebook)	Almost every notebook in the course, by default
Scheduled single-tier	NB 46 — scheduling & orchestration
Three-tier with AI in the middle	NB 34 (POC 2) and the NB 48 capstone
Service-oriented	llm_providers.py — pluggable providers
Microservices	<i>patterns only</i> — too big for a teaching course
ML pipeline	NB 17–19 + NB 45–46 — train / evaluate / serve
ML pipeline, deep-learning variant	NB 20–22 — train / regularise / export (TorchScript)

You build the three-tier and ML-pipeline shapes in **Notebook 34**; the rest you meet as patterns here.

~6 min — close. Point: they have already been using these patterns without naming them. That’s the satisfying reveal.

Note honestly that microservices are *patterns only* in this course — you cannot teach them meaningfully without multiple teams and real traffic, and pretending otherwise would be dishonest.

llm_providers.py is a nice concrete callback: the pluggable-provider shim they’ve used all course *is* a service-oriented boundary — one interface, swappable implementations, no caller changes.

Send them to Notebook 50, then NB 34 to build it. Next lecture: NB 51, AI-assisted software development.